

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический
университет Петра Великого»

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

Направление: 02.03.01 «Математика и компьютерные науки»

Отчет о выполнении лабораторной работы №2
по дисциплине «Математическая логика и теория формальных языков»
«Регулярные выражения»
Вариант №9

Студент,
группы 5130201/30101

_____ Мартыненко А. П.

Преподаватель

_____ Востров А. В.

«_____» _____ 2026г.

Санкт-Петербург, 2026

Содержание

Введение	3
1 Математическое описание	4
1.1 Форматы представления комплексного числа	4
1.2 Регулярные выражения	4
1.3 Регулярное выражение для представления комплексного числа	6
1.4 Конечный автомат	7
1.5 Недетерминированный конечный автомат	9
1.6 Детерминированный конечный автомат	10
2 Особенности реализации	16
2.1 Структуры данных	16
2.2 Реализация класса распознавателя	16
2.2.1 Заполнение таблицы состояний	16
2.2.2 Проверка корректности строки	19
2.2.3 Генерация случайной строки	22
3 Результаты работы программы	24
Заключение	26
Список использованной литературы	27

Введение

В лабораторной работе необходимо построить регулярное выражение, затем недетерминированный конечный автомат и детерминировать его (переходы можно задавать диапазонами). Реализовать программу, которая проверяет введенный текст через реализацию конечного автомата (варианты вывода: строка соответствует, не соответствует, символы не из алфавита). Также необходимо реализовать функцию случайной генерации верной строки по полученному конечному автомату.

Задание для 9 варианта: соответствие комплексного числа разным форматам представления.

1 Математическое описание

1.1 Форматы представления комплексного числа

Комплексное число z может быть представлено в трёх эквивалентных формах: алгебраической, тригонометрической и показательной.

Алгебраическая форма представляется формулой

$$z = a + bi, \quad a, b \in \mathbb{R}, \text{ где}$$

- a — действительная часть;
- b — мнимая часть;
- i — мнимая единица ($i^2 = -1$).

Тригонометрическая форма - представление через модуль r и аргумент φ :

$$z = r(\cos \varphi + i \sin \varphi), \text{ где}$$

- $r = |z| = \sqrt{a^2 + b^2}$ — модуль (расстояние от начала координат);
- $\varphi = \arg(z)$ — аргумент (угол с положительной вещественной осью);
- $\varphi \in (-\pi, \pi]$ — главное значение аргумента.

Показательная (экспоненциальная) форма основана на формуле Эйлера:

$$z = re^{i\varphi}, \quad \text{где } e^{i\varphi} = \cos \varphi + i \sin \varphi$$

Примеры записи комплексных чисел:

$$z = 1 + i = \sqrt{2} \left(\cos \frac{\pi}{4} + i \sin \frac{\pi}{4} \right) = \sqrt{2} e^{i\pi/4}$$

1.2 Регулярные выражения

Регулярные выражения — формальный язык шаблонов для поиска и выполнения манипуляций с подстроками в тексте. Регулярное выражение — это формула, задающая правило поиска подстрок в потоке символов.

Регулярное выражение показывает, как можно построить регулярное множество цепочек из одноэлементных множеств с использованием трех операций: конкатенации, объединения и итерации.

Класс регулярных выражений над конечным словарем Σ определяется так $\emptyset, \varepsilon, a \in \Sigma$ - регулярные выражения.

Если R_1 и R_2 — регулярные выражения, то регулярными выражениями будут:

- их сумма $R_1 + R_2$;
- их произведение (конкатенация) $R_1 \cdot R_2$;
- итерация R_1^* и усеченная итерация R_1^+ (унарные операции).

Приоритеты:

- $*$ итерация;
- $+$ усеченная итерация;
- $\cdot$ произведение (конкатенация);
- $+$ (или $|$) — сложение.

Скобки $()$ — для группировки (задание порядка выполнения операций).

Регулярные множества, как множества цепочек, построенные над конечным словарем (по определенным правилам) — это языки. Их называют регулярными языками.

Правила построения регулярных множеств:

- Объединение R_1 и R_2 обозначается $R_1 \cup R_2$ и состоит из цепочек, которые принадлежат хотя бы одному из множеств R_1 или R_2 .

$$R_1 \cup R_2 = \{\alpha \mid \alpha \in R_1 \text{ или } \alpha \in R_2\}$$

- Конкатенация (произведение) R_1 и R_2 обозначается $R_1 \cdot R_2$ и состоит из цепочек, которые можно разбить на две части, одна из которых принадлежит R_1 , а другая R_2 .

$$R_1 \cdot R_2 = \{\alpha\beta \mid \alpha \in R_1 \text{ и } \beta \in R_2\}$$

Обозначим $R^0 = \{\varepsilon\}$, $R^1 = R$, $R^2 = R \cdot R$, $R^{k+1} = R^k \cdot R$, и так далее. Обозначение ε обозначает пустую цепочку.

- Итерация R обозначается R^* и состоит из цепочек, которые можно разбить на произвольное количество повторений множества R .

$$R^* = \{\varepsilon\} \cup R \cup R^2 \cup R^3 \cup \dots$$

- Итерация Клини (R^*): множество цепочек, состоящих из нуля или более повторений цепочек из R .
- Усечённая итерация (R^+): аналогична итерации, но требует минимум одного повторения ($R^+ = RR^*$)

1.3 Регулярное выражение для представления комплексного числа

Для валидации и генерации разных форматов представления комплексного числа используется составное регулярное выражение `(?:exponential|trig|algebraic)`, где `?:` означает группировку без сохранения результата в отдельную переменную, а `exponential`, `trig`, `algebraic` - форматы представления комплексного числа (экспоненциальное, тригонометрическое и алгебраическое). Знак `|` означает, что регулярное выражение может распознать любую из трех представленных форм.

Для улучшения читаемости в регулярных выражениях используются базовые выражения:

`real_number = (?: 0(?:\.\d+)?|[1-9]\d*(?:\d+)?` - для отображения базового числа 3 , 0.5 , 3.88 . Структура:

- `?:` - группа без захвата
- `0(?:\d+)?` - ноль и опциональная дробная часть (точка и одна или более цифра). Дробная часть может отсутствовать
- `|` - или
- `[1-9]\d*(?:\d+)?` - первая цифра от 1 до 9, а потом 0 или более цифр после первой. Также опциональная дробная часть (точка и одна или более цифр).

`number = (?:[-]?{real_number} | [-]?sqrt({real_number}))` - для отображения вещественного числа вида $number$ или $sqrt(number)$. Примеры: $sqrt{3}$, $-sqrt{0.5}$, -5.66 . Структура:

- `[-]?{real_number}` - опциональный знак минус и обычное число
- `[-]? sqrt({real_number}` - опциональный знак минус, ключевое слово `sqrt`, открывающая скобка, число вида `real_bumber`, закрывающая скобка (пример - `sqrt(3.5)`)

`arg = (?:{number}|pi/({number}))` - аргумент для функций вида $number$ или $pi/(number)$. Примеры: 3 , $pi/0.5$, $pi/3.88$. Структура:

- `{number}` - вещественное число
- `pi/({number})` - ключевое слово `pi/`, открывающая скобка, число вида `number`, закрывающая скобка

Регулярные выражения для основных форматов:

`algebraic = (?:{number}?i|{number}\s?[+-]\s?{number}?i )` - алгебраический формат представления (вид bi или $a \pm bi$). Структура:

- $\{number\}?i$ - опциональное вещественное число и обязательное i
- $|$ - или
- $number\s? [+ -] \s? number?i$ - вещественное число, ноль или один пробел, плюс или минус, ноль или один пробел, опциональное вещественное число с обязательным i в конце

$exponential = (?:\{number\})\s? \[*? exp*(i*\{arg\})$ - экспоненциальный формат представления (вид $exp(i*\{arg\})$ или $\{number\} exp(i*\{arg\})$). Структура:

- $\{number\}\s? \[*?$ - опциональное вещественное число (множитель), ноль или один пробел, ноль или один знак умножения
- $exp*(i*arg)$ - обязательная часть: экспонента, открывающая скобка, i^* , аргумент функции.

$trig = (? : number\s? \[*?cos(\{arg\})\s? [+ -] \s? i*sin(\{arg\})$ - тригонометрический формат представления (вид $number(cos(arg) \pm i*sin(arg))$). Структура:

- $\{number\}\s? \[*?$ - опциональное вещественное число (множитель), ноль или один пробел, ноль или один знак умножения
- $cos(\{arg\})\s? [+ -] \s? i*sin(\{arg\})$ - обязательная часть: косинус, аргумент функции, ноль или один пробел, плюс или минус, ноль или один пробел, i^* синус, аргумент функции.

Согласно теореме Клини, классы регулярных множеств и автоматных языков совпадают. Это значит, что:

- Любой язык, распознаваемый конечным автоматом, может быть задан регулярным выражением.
- Для любого регулярного выражения существует конечный автомат, распознающий соответствующий язык.

Следовательно, можно построить алгоритм распознавания и преобразования цепочек – трансляцию – как имитацию работы детерминированного конечного автомата.

1.4 Конечный автомат

Конечный автомат - это математическая модель, которая определяется набором $A = (S, \Sigma, F, S_0, \delta)$, где:

- $S = \{S_0, \dots, S_{35}\}$ - конечное множество состояний;
- $\Sigma = \{0123456789+^*/().isqrtexpcon\langle ws \rangle\}$ - конечное множество входных сигналов;
- $F = \{S_{18}, S_{19}, S_{20}\}$ - конечное множество финальных состояний;
- S_0 - начальное состояние ($S_0 \in S$);
- $\delta : S \times \Sigma \rightarrow S$ - функция переходов.

В момент времени $t=0$ автомат находится в начальном состоянии s_0 . Конечный автомат работает в дискретные моменты времени.

Конечный автомат-распознаватель используется для распознавания цепочек символов в соответствии с заданным формальным языком. Конечный автомат-распознаватель является распознающей грамматикой.

Автомат A допускает (распознает) цепочку, если эта цепочка переводит A из начального в одно из финальных состояний. Автомат A допускает язык L , если он допускает все цепочки этого языка – и только их.

Выделяют недетерминированный и детерминированный конечные автоматы.

Недетерминированный конечный автомат-распознаватель $A = (S, \Sigma, S_0, \delta, F)$, где:

- S – конечное множество состояний
- Σ – конечное множество входов
- $S_0 \subseteq S$ – множество начальных состояний
- $\delta : S \times \Sigma \rightarrow 2^S$ – функция переходов
- $F \subseteq S$ – множество финальных состояний

В недетерминированном конечном автомате-распознавателе могут быть следующие неоднозначности:

- несколько начальных состояний,
- несколько переходов, помеченных одним и тем же символом,
- переходы, помеченные пустым символом ε .

1.5 Недетерминированный конечный автомат

По регулярному выражению был построен недетерминированный автомат, который представлен на рисунке 1.

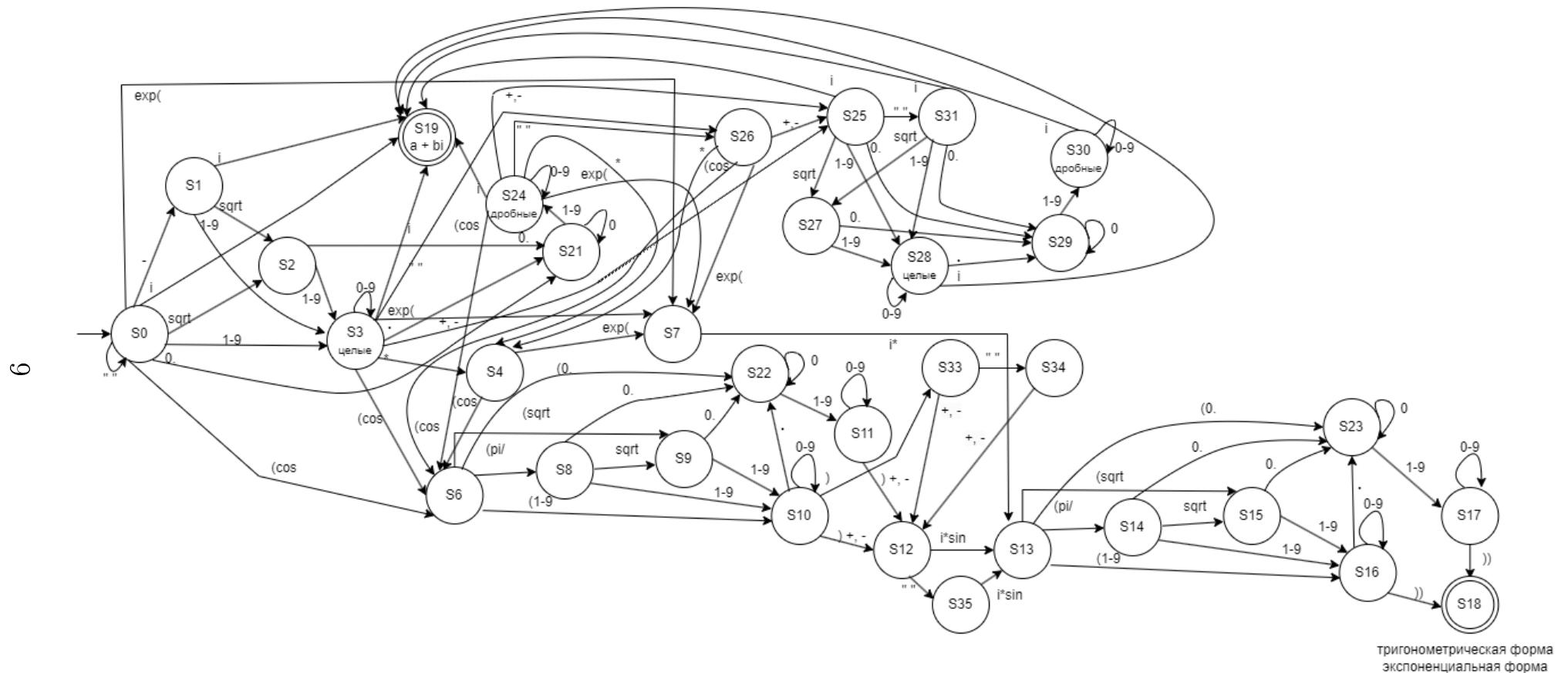


Рис. 1. Недетерминированный конечный автомат для представления комплексного числа

Таблица 1. Таблица переходов детерминированного конечного автомата

Текущее состояние	Входящий токен	Следующее состояние
S0	-	S1
	i	S19 (final - success)
	sqrt	S2
	1-9	S3
	cos	S6
	exp(	S7
	0.	S21
	«whitespace»	S0
	other	S20 (final - error)
S1	i	S19 (final - success)
	sqrt	S2
	1-9	S3
	other	S20 (final - error)
S2	1-9	S3
	0.	S21
	other	S20 (final - error)
S3	0-9	S3
	1-9	S3
	i	S19 (final - success)
	.	S21
	«whitespace»	S4
	+	S25
	-	S25
	*	S4
	(cos	S6
	exp(	S7
	other	S20 (final - error)
S4	exp(	S7

Продолжение таблицы 1

Текущее состояние	Входящий токен	Следующее состояние
	(cos	S6
	other	S20 (final - error)
S6	(sqrt	S9
	(pi/	S8
	(1-9	S10
	(0.	S22
	other	S20 (final - error)
S7	i*	S13
	other	S20 (final - error)
S8	1-9	S10
	sqrt	S9
	0.	S22
	other	S20 (final - error)
S9	1-9	S10
	0.	S22
	other	S20 (final - error)
S10	0-9	S10
	1-9	S10
	.	S22
	) +	S12
	) -	S12
	)	S33
	other	S20 (final - error)
S11	0-9	S11
	1-9	S11
	) +	S12
	) -	S12
	other	S20 (final - error)

Продолжение таблицы 1

Текущее состояние	Входящий токен	Следующее состояние
S12	i*sin	S13
	«whitespace»	S35
	other	S20 (final - error)
S13	(sqrt	S15
	(pi/	S14
	(1-9	S16
	(0.	S23
	other	S20 (final - error)
S14	sqrt	S15
	1-9	S16
	0.	S23
	other	S20 (final - error)
S15	1-9	S16
	0.	S23
	other	S20 (final - error)
S16	0-9	S16
	1-9	S16
	.	S23
	))	S18 (final - success)
	other	S20 (final - error)
S17	0-9	S17
	1-9	S17
	))	S18 (final - success)
	other	S20 (final - error)
S21	0-9	S21
	1-9	S24
	other	S20 (final - error)
S22	0-9	S22

Продолжение таблицы 1

Текущее состояние	Входящий токен	Следующее состояние
	1-9	S11
	other	S20 (final - error)
S23	0-9	S23
	1-9	S17
	other	S20 (final - error)
S24	0-9	S24
	1-9	S24
	i	S19 (final - success)
	*	S4
	«whitespace»	S4
	+	S25
	-	S25
	(cos	S25
	exp(	S25
	other	S20 (final - error)
S25	1-9	S28
	i	S19 (final - success)
	sqrt	S27
	«whitespace»	S4
	0.	S29
	other	S20 (final - error)
S26	*	S4
	+	S25
	-	S25
	(cos	S6
	exp(	S7
	other	S20 (final - error)
S27	1-9	S28

Продолжение таблицы 1

Текущее состояние	Входящий токен	Следующее состояние
	0.	S29
	other	S20 (final - error)
S28	0-9	S28
	1-9	S28
	.	S29
	i	S19 (final - success)
	other	S20 (final - error)
S29	1-9	S30
	0-9	S29
	other	S20 (final - error)
S30	1-9	S30
	0-9	S30
	i	S19 (final - success)
	other	S20 (final - error)
S31	1-9	S28
	sqrt	S27
	0.	S29
	i	S19 (final - success)
	other	S20 (final - error)
S33	+	S12
	-	S12
	«whitespace»	S34
	other	S20 (final - error)
S34	+	S12
	-	S12
	other	S20 (final - error)
S35	i*sin	S13
	other	S20 (final - error)

2 Особенности реализации

2.1 Структуры данных

Лексический анализатор представлен следующими структурами данных:

1. `alphabet` - множество допустимых символов входного языка.
2. `start_state`, `error_state`, `current_state` - строки, обозначающие начальное, конечное ошибочное, текущее состояние детерминированного конечного автомата.
3. `accept_states` - множество допускающих (финальных) состояний автомата.
4. `transitions` - словарь, реализующий таблицу переходов ДКА. Структура имеет вид: `{состояние: {токен: следующее_состояние}}` Ключами внешнего словаря являются состояния автомата. Каждому состоянию соответствует внутренний словарь, где ключами выступают токены, а значениями - состояния, в которые осуществляется переход при поступлении данного токена.
5. `token_to_text` - словарь, используемый при генерации случайных строк.
6. `state_history` - список, накапливающий последовательность состояний, пройденных автоматом при анализе строки/

2.2 Реализация класса распознавателя

Класс `Regs` представляет структуру, которая обрабатывает входящую строку, проверяет ее корректность, генерирует случайную строку по заданному конечному автомату и хранит таблицу переходов и состояний.

2.2.1 Заполнение таблицы состояний

Метод `__init__` предназначен для инициализации и хранения входного алфавита, стартового и финальных состояний, а также таблицы переходов и состояний.

Вход: ссылка на экземпляр класса.

Выход: таблица переходов и состояний, стартовое и финальные состояния, входной алфавит.

Листинг 1. Функция `__init__`

```
1 def __init__(self):  
2 self.alphabet = set("0123456789+*/().isqrtexpco sin pi ")
```



```

3 self.start_state = "S0"
4 self.accept_states = {"S18", "S19"}
5 self.error_state = "S20"
6 self.transitions = {
7     "S0": {
8         "-": "S1",
9         "i": "S19",
10        "sqrt": "S2",
11        "digit19": "S3",
12        "cos": "S6",
13        "exp(": "S7",
14        "0.": "S21"
15    },
16
17    "S1": {
18        "i": "S19",
19        "sqrt": "S2",
20        "digit19": "S3"
21    },
22
23    "S2": {
24        "digit19": "S3",
25        "0.": "S21"
26    },
27
28    "S21": {
29        "digit": "S21",
30        "digit19": "S24"
31    },
32
33    "S3": {
34        "digit": "S3",
35        "digit19": "S3",
36        "i": "S19",
37        ".": "S21",
38        "space": "S4"
39    },
40
41    "S4": {
42        "exp(": "S7",
43        "cos": "S6",
44        "+": "S0",
45        "-": "S0"
46    },
47
48
49
50    "S24": {
51        "digit": "S24",
52        "digit19": "S24",
53        "i": "S19",
54        "space": "S4"
55    },
56
57    "S6": {
58        "(sqrt": "S9",
59        "(pi/": "S8",
60        "(digit19": "S10",
61        "(0.": "S22"
62    },

```

```

63
64 "S7": {
65     "i*": "S13"
66 },
67
68 "S8": {
69     "digit19": "S10",
70     "sqrt": "S9",
71     "0.": "S22"
72 },
73
74 "S9": {
75     "digit19": "S10",
76     "0.": "S22"
77 },
78
79 "S22": {
80     "digit": "S22",
81     "digit19": "S11"
82 },
83
84 "S10": {
85     "digit": "S10",
86     "digit19": "S10",
87     ".": "S22",
88     ")+" : "S12",
89     ")-" : "S12"
90 },
91
92 "S11": {
93     "digit": "S11",
94     "digit19": "S11",
95     ")+" : "S12",
96     ")-" : "S12"
97 },
98
99 "S12": {
100     "i*sin": "S13"
101 },
102
103 "S13": {
104     "(sqrt": "S15",
105     "(pi/" : "S14",
106     "(digit19": "S16",
107     "(0.": "S23"
108 },
109
110 "S14": {
111     "sqrt": "S15",
112     "digit19": "S16",
113     "0.": "S23"
114 },
115
116 "S15": {
117     "digit19": "S16",
118     "0.": "S23"
119 },
120
121 "S23": {
122     "digit": "S23",

```

```

123     "digit19": "S17"
124 },
125
126 "S16": {
127     "digit": "S16",
128     "digit19": "S16",
129     ".": "S23",
130     ")": "S18"
131 },
132
133 "S17": {
134     "digit": "S17",
135     "digit19": "S17",
136     ")": "S18"
137 }
138
139 }

```

2.2.2 Проверка корректности строки

Метод `check_alphabet` предназначен для проверки введенной строки на наличие символов, не принадлежащих входному языку.

Вход: изначальный текст.

Выход: флаг, показывающий, есть ли некорректные символы в строке; некорректные символы.

Листинг 2. Функция `check_alphabet`

```

1 def check_alphabet(self, text):
2     invalid = set(text) - self.alphabet
3     return len(invalid) == 0, invalid

```

Метод `tokenize` разбиения входной строки на токены. Пока строка не обработана полностью, проверяется наличие известных токенов в начале остатка строки. При совпадении соответствующий токен добавляется в список, а остаток строки сокращается на длину совпавшего токена. Если не подошел ни один из шаблонов, то символу присваивается токен «other».

Вход: изначальный текст.

Выход: заполненный массив токенов.

Листинг 3. Функция `tokenize`

```

1     def tokenize(self, text):
2
3     tokens = []
4     remaining = text
5     while len(remaining) > 0:
6         if remaining.startswith("i*sin"):
7             tokens.append("i*sin")
8             remaining = remaining[5:]
9             continue
10        if remaining.startswith("(sqrt)":
11            tokens.append("(sqrt)")
12            remaining = remaining[5:]
13            continue
14        if remaining.startswith("sqrt"):

```

```

15 tokens.append("sqrt")
16 remaining = remaining[4:]
17 continue
18 if remaining.startswith("(cos)":
19 tokens.append("cos")
20 remaining = remaining[4:]
21 continue
22 if remaining.startswith("exp("):
23 tokens.append("exp(")
24 remaining = remaining[4:]
25 continue
26 if remaining.startswith("(pi/"):
27 tokens.append("(pi/")
28 remaining = remaining[4:]
29 continue
30 if remaining.startswith("(0."):
31 tokens.append("(0.")
32 remaining = remaining[3:]
33 continue
34 if remaining.startswith("cos)":
35 tokens.append("cos")
36 remaining = remaining[3:]
37 continue
38 if remaining.startswith("i*"):
39 tokens.append("i*")
40 remaining = remaining[2:]
41 continue
42 if remaining.startswith(")+"):
43 tokens.append(")+")
44 remaining = remaining[2:]
45 continue
46 if remaining.startswith("0."):
47 tokens.append("0.")
48 remaining = remaining[2:]
49 continue
50 if remaining.startswith(")-"):
51 tokens.append(")-")
52 remaining = remaining[2:]
53 continue
54 if len(remaining) >= 2 and remaining[1].isdigit() and remaining[1] != '0'
    and remaining[0] == '(':
55 tokens.append("(digit19")
56 remaining = remaining[2:]
57 continue
58 c = remaining[0]
59 if c.isspace():
60 tokens.append("space")
61 remaining = remaining[1:]
62 continue
63 if c in "+-":
64 tokens.append(c)
65 remaining = remaining[1:]
66 continue
67 if c == ".":
68 tokens.append(".")
69 remaining = remaining[1:]
70 continue
71 if c == ")":
72 tokens.append(")")
73 remaining = remaining[1:]

```

```

74 continue
75 if c == "(":
76     tokens.append("(")
77     remaining = remaining[1:]
78     continue
79 if c == "i":
80     tokens.append("i")
81     remaining = remaining[1:]
82     continue
83 if c.isdigit():
84     if c == "0":
85         tokens.append("digit")
86     else:
87         tokens.append("digit19")
88         remaining = remaining[1:]
89         continue
90 tokens.append("other")
91 remaining = remaining[1:]
92 return tokens

```

Метод `step` предназначен для реализации одного шага работы детерминированного конечного автомата. На основе текущего состояния и входного токена определяется следующее состояние в соответствии с таблицей переходов. Если такого состояния или токена нет в таблице состояний и переходов, то возвращается ошибка.

Вход: текущее состояние, токен.

Выход: следующее состояние.

Листинг 4. Функция `step`

```

1 def step(self, state, token):
2 if state not in self.transitions:
3     return self.error_state
4 if token not in self.transitions[state]:
5     return self.error_state
6 return self.transitions[state][token]

```

Метод `validate` предназначен для проверки корректности входного текста через конечный автомат. Сначала с помощью вспомогательной функции `check_alphabet` проверяется, что все символы принадлежат входному алфавиту, потом текст токенизируется функцией `tokenize`, а затем начиная с начального состояния пошагово проходит по конечному автомату. Если на каком-то этапе состояние стало ошибочным, то выводится результат «строка не соответствует», если дошли до корректного финального состояния - выводится результат «строка соответствует».

Вход: входной текст.

Выход: результат проверки корректности текста.

Листинг 5. Функция `validate`

```

1 def validate(self, text: str) -> str:
2 is_valid_alphabet, invalid_chars = self.check_alphabet(text)
3 if not is_valid_alphabet:
4     return f"символы не из алфавита: {invalid_chars}\n"
5 text = text.strip()

```

```

6 if not text:
7     return "строка не соответствует"
8 tokens = self.tokenize(text)
9 print(tokens)
10 current_state = self.start_state
11 print(current_state, end=" ")
12 state_history = [current_state]
13 for i, token in enumerate(tokens):
14     next_state = self.step(current_state, token)
15     print(next_state, end=" ")
16     if next_state == self.error_state:
17         print(" ")
18         return f"строка не соответствует ошибка( на токене {i}: '{token}')\n"
19     current_state = next_state
20     state_history.append(next_state)
21 print()
22 if current_state in self.accept_states:
23     return "строка соответствует\n"
24 else:
25     return f"строка не соответствует финальное( состояние: {current_state})\n"

```

2.2.3 Генерация случайной строки

Метод `generate_random_string` предназначен для генерации случайной корректной строки. Для этого задается словарь, в котором ключом является токен, а значением - текст этого токена. Начиная с начального состояния, программа идет по конечному автомату, в каждом состоянии рандомно выбирая один из допустимых вариантов перехода. В соответствии с токеном в строку добавляется нужный текст. Переходы происходят до тех пор, пока программа не пройдет конечный автомат до финального состояния.

Вход: экземпляр класс.

Выход: сгенерированная случайная корректная строка.

Листинг 6. Функция `generate_random_string`

```

1 def generate_random_string(self) -> str:
2     token_to_text = {
3         "digit19": lambda: str(random.randint(1, 9)),
4         "digit": lambda: "0", #str(random.randint(0, 9)),
5         "0.": lambda: "0.",
6         ".": lambda: ".",
7         "+": lambda: "+",
8         "-": lambda: "-",
9         "*": lambda: "*",
10        "space": lambda: " ",
11        "i": lambda: "i",
12        "sqrt": lambda: "sqrt",
13        "1*exp(": lambda: "1*exp(",
14        "1*(cos": lambda: "1*(cos",
15        "(sqrt": lambda: "(sqrt",
16        "(cos": lambda: "(cos",
17        "cos": lambda: "cos",
18        "(pi/": lambda: "(pi/",
19        "(digit19": lambda: "(" + str(random.randint(1, 9)),

```

```

20     "(0.": lambda: "(0.",
21     "exp(": lambda: "exp(",
22     "i*": lambda: "i*",
23     "i*sin": lambda: "i*sin",
24     ")+" : lambda: ")+",
25     ")-" : lambda: ")-",
26     ")" : lambda: ")",
27     "(" : lambda: "(",
28     "))" : lambda: ")),
29 }
30 state = self.start_state
31 result = ""
32 while True:
33     if state in self.accept_states and random.random() < 0.4:
34         break
35     if state not in self.transitions:
36         break
37     possible_tokens = list(self.transitions[state].keys())
38     token = random.choice(possible_tokens)
39     if token in token_to_text:
40         result += token_to_text[token]()
41     else:
42         result += token
43     state = self.transitions[state][token]
44     if state == self.error_state:
45         break
46     return result
47

```

3 Результаты работы программы

На рисунках 3- 4 показана проверка введенной строки на соответствие формату комплексных чисел.

```
Examples of available formats:
1. Algebraic: 3+4i, 0.5-1.7i, -sqrt3+4i
2. Exponential: exp(i*(pi/4)), exp(i*(1.57)), sqrt3.14*exp(i*(pi/2)), 3 exp(i*(pi/6))
3. Trigonometric: (cos(pi/4)+i*sin(pi/4)), 5.34*(cos(pi/4)+i*sin(pi/4)), 5.34(cos(pi/4)+i*sin(pi/4))
If you want to go back to main menu, input 'back'

Input complex number: -sqrt3+4i
['-', 'sqrt', 'digit19', '+', 'digit19', 'i']
S0 S1 S2 S3 S25 S28 S19
Result: строка соответствует

Input complex number: 3 exp(i*(pi/6))
['digit19', 'space', 'exp(', 'i*', '(pi/', 'digit19', ')')']
S0 S3 S26 S7 S13 S14 S16 S18
Result: строка соответствует

Input complex number: 5.34*(cos(pi/4)+i*sin(pi/4))
['digit19', '.', 'digit19', 'digit19', '*', '(cos', '(pi/', 'digit19', ')+', 'i*sin', '(pi/', 'digit19', ')')']
S0 S3 S21 S24 S24 S4 S6 S8 S10 S12 S13 S14 S16 S18
Result: строка соответствует
```

Рис. 3. Корректно введенные примеры

```
Input complex number: 3+4#i
Result: символы не из алфавита: {'#'}

Input complex number: 44
['digit19', 'digit19']
S0 S3 S3
Result: строка не соответствует (финальное состояние: S3)

Input complex number: (cos(pi/4)+i*sin(pi/))
['(cos', '(pi/', 'digit19', ')+', 'i*sin', '(pi/', ')']
S0 S6 S8 S10 S12 S13 S14 S20
Result: строка не соответствует (ошибка на токене 6: ')')

Input complex number:
строка не соответствует (пустая строка)
```

Рис. 4. Некорректно введенные примеры

На рисунке 5 показана генерация рандомной строки, удовлетворяющей формату комплексных чисел.

```
['exp(', 'i*', '(pi/', 'sqrt', 'digit19', '))']  
S0 S7 S13 S14 S15 S16 S18  
Random:  exp(i*(pi/sqrt4)) -> строка соответствует  
  
['sqrt', '0.', 'digit19', 'i']  
S0 S2 S21 S24 S19  
Random:  sqrt0.2i -> строка соответствует  
  
['digit19', '(cos', '(pi/', 'sqrt', '0.', 'digit', 'digit', 'digit19',  
S0 S3 S6 S8 S9 S22 S22 S22 S11 S11 S12 S13 S23 S17 S17 S17 S18  
Random:  5(cos(pi/sqrt0.0062)+i*sin(0.305)) -> строка соответствует
```

Рис. 5. Сгенерированные строки

Заключение

В ходе выполнения лабораторной работы были построены регулярное выражение и конечный автомат, которые распознают комплексное число в трех форматах. Конечный автомат был детерминирован, и на его основе была разработана программа для проверки корректности введенного комплексного числа, а также генерации случайного комплексного числа, распознаваемого конечным автоматом. Реализованная программа была протестирована на наборе примеров, включающем как корректные, так и некорректные входные данные.

Согласно теореме Клини, класс языков, распознаваемых конечными автоматами, совпадает с классом регулярных языков. Разработанное регулярное выражение для комплексных чисел является конечным автоматом в другой форме, что подтверждает их эквивалентность.

Достоинства программы:

- простота модификации таблицы переходов и выходов;
- токенизация «жадным алгоритмом», что позволяет корректно выделить все токены.

Недостатки программы:

- при генерации невозможно задать количество цифр после запятой, из-за этого числа получаются длинными;
- строгая последовательность компонентов комплексного числа.

Масштабирование программы:

- использование логирования вместо выводов ошибок через функцию `print`;
- поддержка альтернативных форматов комплексных чисел, например, матричной формы:

$$\begin{pmatrix} a & -b \\ b & a \end{pmatrix}$$

- увеличение «гибкости» распознавания за счет работы с пробелами: распознавание чисел вида
 - $3 * \exp(i * (pi/6))$
 - $3 * \exp(i * (pi/6))$
 - $3 * \exp(i * (pi/6))$

Список использованной литературы

- [1] Секция «Телематика». — Текст : электронный // tema.spbstu.ru : [сайт]. URL: <https://tema.spbstu.ru/mathem/> (дата обращения: 10.03.2026).
- [2] Сети, Р.; Ахо, А. Компиляторы: принципы, технологии и инструментарий - М.: Издательство «Наука», 2008. - 1178 с. (дата обращения: 10.03.2026).
- [3] Ю.Г. Карпов Теория автоматов. - СПб: Издательский дом «Питер», 2003. - 208 с. (дата обращения - 11.03.2026).